Chapter 21: Functional Programming

By Irina Galata

The evolution of programming as an engineering discipline includes improvement of languages and tools. There are also different fundamental approaches you can use to develop your programs, often called **paradigms**. There are various programming paradigms, but don't be intimidated, since all of them have strengths and weaknesses.

The more comfortable you become with the different approaches, the easier it will be able to apply them. Plus, you already know at least one of them: object-oriented programming aka OOP. In this chapter, you'll get acquainted with another type of programming — **functional programming** — and learn its technical details.

What is functional programming?

You may remember that a key feature of OOP is that classes and their instances contain properties and methods. Functional programming is instead based around the use of functions, which ideally don't have **side effects**.

A side effect is any change to the state of a system. A simple example of a side effect is printing something to the screen. Another is changing the value of the property of an object. Side effects are typically rampant in OOP, as class instances send messages back and forth to one another and change their internal state.

Another key feature of functional programming making functions **first-class** citizens of the language, as you'll see in a future section.

Most functional programming languages also rely on a concept called **referential transparency**. This term effectively means that given the same input, a function will always return the same output. When a function has this property it is called a **pure function**.

Functions in functional programming languages are more like their mathematical namesake functions than typical functions in non-functional programming approaches.

Unlike versions of Java prior to Java 8, Kotlin allows you to use both the OOP and functional approaches to building software, either separately or by combining them to make your code more efficient and flexible. In this chapter, you'll see an example of the combination of the ideas of OOP and functional programming.

Getting started

Before diving into using functional programming, you'll setup a system that will let you explore the details. Open up the starter project and keep reading.

In this chapter, you're going to write a program to conduct a battle between two robots. First of all, you need to create those battle robots.

Create a class called `Robot` with the following definition:

```
import java.util.*

class Robot(val name: String) {
  var strength: Int = 0
```

```
  private var health: Int = 100

  init {
    strength = Random().nextInt(100) + 10
    report("Created (strength $strength)")
  }

  fun report(message: String) {
    println("$name: \t$message")
  }
}
```

You've created a robot class for robots that have some amount of strength and health and can report messages.

To take part in a battle, your robot should be able to take damage from another robot. Add the following code to the `Robot` class:

```
// 1
var isAlive: Boolean = true

private fun damage(damage: Int) {
  // 2
  val blocked = Random().nextBoolean()

  if (blocked) {
    report("Blocked attack")
    return
  }

  // 3
  health -= damage
  report("Damage -$damage, health $health")

  // 4
  if (health <= 0) {
    isAlive = false
  }
}
```

Here's what's going on above:

1. The `isAlive` property keeps track of if a robot is able to continue the battle.
2. In the `damage()` function, you give a robot a chance to block the attack of another robot using the `nextBoolean()` function on a new `Random()`.
3. If an attacked robot couldn't defend itself, you decrease its health.
4. And, finally, you check to see if it's still alive after the attack.

Your robot also needs to be able to cause damage to another robot. Add the following code to the `Robot` class:

```
// 1
fun attack(robot: Robot) {
  // 2
  val damage = (strength * 0.1 + Random().nextInt(10)).toInt()

  // 3
  robot.damage(damage)
}
```

In this code you:

1. Define the `attack()` function, which receives another robot as an argument.
2. Calculate a damage value depending on the strength of the current robot and its luck.
3. Do damage to the other robot.

Now that you have a robot that can cause damage, your robots need a space to conduct the battle.

Next, create a `Battlefield` object:

```
object Battlefield {
  // 1
  fun beginBattle(firstRobot: Robot, secondRobot: Robot) {
    // 2
    var winner: Robot? = null
    // 3
    battle(firstRobot, secondRobot)
    // 4
    winner = if (firstRobot.isAlive) firstRobot else secondRobot
  }

  fun battle(firstRobot: Robot, secondRobot: Robot) {
    // 5
    firstRobot.attack(secondRobot)
    // 6
    if (secondRobot.isAlive.not()) {
      return
    }
    // 7
    secondRobot.attack(firstRobot)

    if (firstRobot.isAlive.not()) {
      return
    }
```

```
      // 8
      battle(firstRobot, secondRobot)
    }
  }
```

Above, you do the following:

1. Declare a function that will initiate a battle between two participants.
2. Declare a variable to define the winner of the fight.
3. Perform the battle.
4. Check which robot has won.
5. In `battle()`, force the first robot to attack the second one.
6. Check if the second robot is alive. If not, finish the fight.
7. Repeat the previous steps for the second robot, letting it fight back.
8. Call the `battle()` function from itself to continue the battle while the robots are still alive.

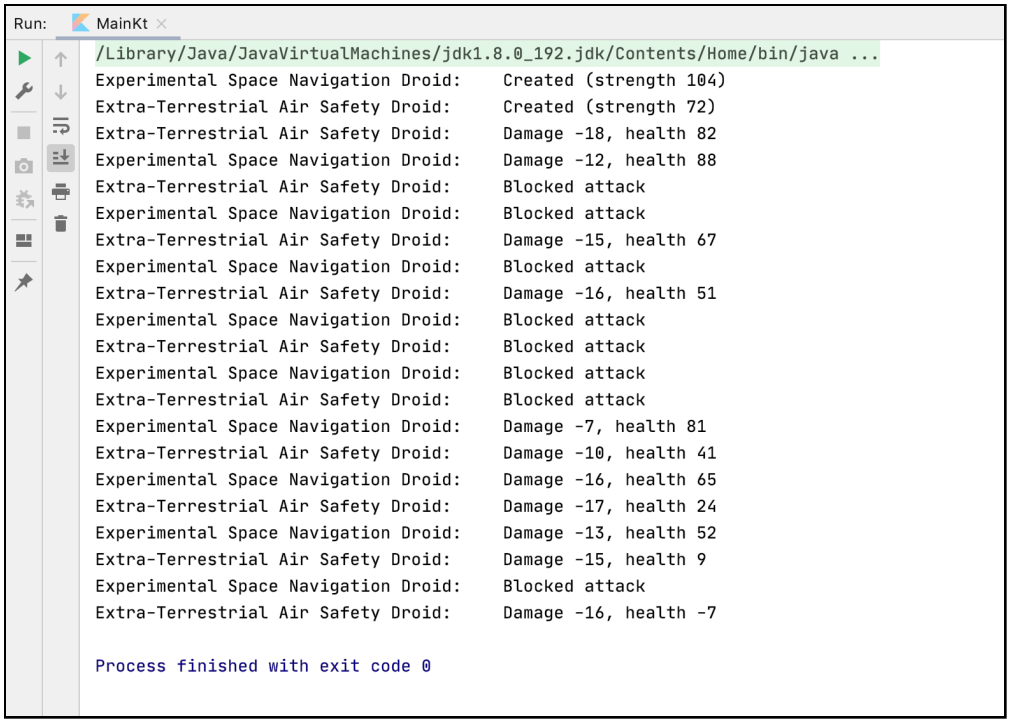
The `battle()` function is an example of a **recursive function**: a function that calls itself. Recursive functions are common in strict functional programming languages, since they are used to replace loops. Recursive functions are susceptible to a condition known as **stack overflow**, where the function call stack exceeds a limit, if they call themselves too many times. You'll see near the end of chapter how, in certain cases, you can avoid stack overflow in Kotlin while still coding with recursive functions.

To create a battle, in the `main()` function, add the following lines of code:

```
val firstRobot = Robot("Experimental Space Navigation Droid")
val secondRobot = Robot("Extra-Terrestrial Air Safety Droid")

Battlefield.beginBattle(firstRobot, secondRobot)
```

Your robots are ready to fight! Run the application. You'll get a similar output:



From the output above, it looks like "Experimental Space Navigation Droid" won.

First-class and higher-order functions

One of the main ideas of functional programming is **first-class** functions. This means that you can operate with functions in the same ways you can other elements of the language — you can pass functions as arguments to other functions, return functions from functions, and assign functions to a variable. Functions that receive a function as a parameter or return functions are called **higher-order** functions.

Function types

To declare a function which receives a function parameter or returns another function, it's necessary to know how to specify a **function type**.

As an example, a function of type `(Int, Int) -> Float` receives two `Int` parameters and returns a `Float`. In parentheses, you define the types of the function parameters separated by a comma. After the –> symbol, you give the function return type. This function type would be read as something like "Int, Int to Float".

The function type for a function that takes no parameters and returns no meaningful value is `() -> Unit` in Kotlin.

Passing a function as an argument

Update `beginBattle()` to receive another function as a parameter, which will be executed when the battle is finished. That way, you'll know exactly which robot has won:

```
fun beginBattle(
  firstRobot: Robot,
  secondRobot: Robot,
  onBattleEnded: (Robot) -> Unit
) {
  var winner: Robot? = null
  battle(firstRobot, secondRobot)
  winner = if (firstRobot.isAlive) firstRobot else secondRobot
  onBattleEnded(winner)
}
```

As you see, `beginBattle()` now receives `onBattleEnded`, a function of type `(Robot) -> Unit`, which means that it receives an instance of `Robot` and returns `Unit`. Once the winner is known, you invoke it by passing a robot `winner` to `onBattleEnded()`.

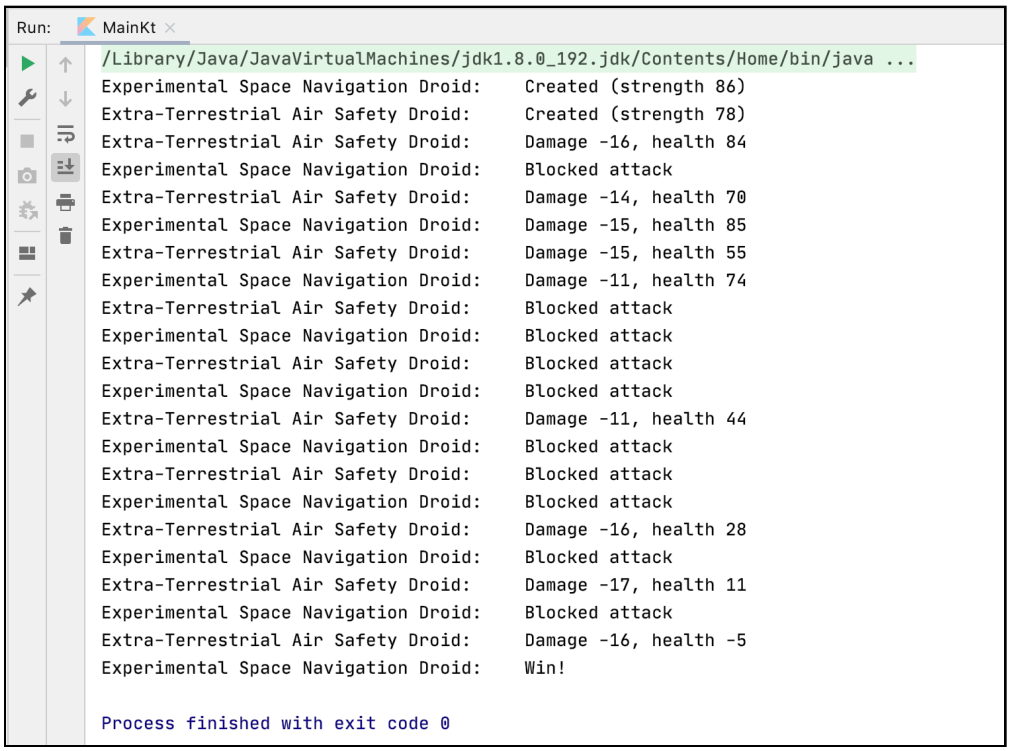
Update the `main()` function and add `onBattleEnded()`:

```
fun main() {
  val firstRobot = Robot("Experimental Space Navigation Droid")
  val secondRobot = Robot("Extra-Terrestrial Air Safety Droid")
  Battlefield
    .beginBattle(firstRobot, secondRobot, ::onBattleEnded)
}

fun onBattleEnded(winner: Robot) {
  winner.report("Win!")
}
```

To pass a named function as an argument to another function, you use the `::` operator.

Run the app again.



Now, you can see the winner of the battle directly.

Returning functions

Similar to passing functions as arguments, you can return a function from another function, as in the following code:

```
fun someFunction(): () -> Int {
  return ::anotherFunction
}

fun anotherFunction(): Int {
  return Random().nextInt()
}
```

`someFunction()` returns a function of type `() -> Int`, which fits `anotherFunction()`, so you can return `anotherFunction` from `someFunction()` using the `::` operator.

Lambdas

As you learned about in Chapter 10, a **lambda** is a function literal, which can be invoked, passed as an argument or returned just like ordinary functions. In this chapter, you'll learn a bit more about lambdas in the context of functional programming.

Recall the lambda syntax, using a lambda assigned to a variable `pow`:

```
val pow = { base: Int, exponent: Int ->
  Math.pow(base.toDouble(), exponent.toDouble())
}
```

A lambda expression is always defined in curly brackets. First, you declare the names and types of the lambda parameters, and, after the –> sign, you place the body of your lambda.

You don't have to use the `return` keyword inside a lambda, nor do you have to specify its return type. The last expression in a lambda body determines the return type and the value that is returned — `Math.pow(base.toDouble(), exponent.toDouble())` of type `Double` in this case.

Once defined into a variable, you can use a lambda by calling it as if it were a function:

```
pow(2, 4)
```

There's also another way to declare a lambda:

```
val pow: (Int, Int) -> Double = { base, exponent ->
  Math.pow(base.toDouble(), exponent.toDouble())
}
```

You can explicitly declare the type of a lambda but not have to specify the types of parameters inside the brackets. Just like in the previous example, the lambda receives two parameters of type `Int` and returns a `Double`.

If a lambda has only one parameter, you don't need to specify its name. You can access it by using `it` as a name:

```
val root: (Int) -> Double = { Math.sqrt(it.toDouble()) }
```

Using a lambda, you can update your call to `beginBattle()` in the `main()` function in the following way:

```
val onBattleEnded = { winner: Robot -> winner.report("Win!") }
Battlefield.beginBattle(firstRobot, secondRobot, onBattleEnded)
```

Or even more conveniently:

```
Battlefield.beginBattle(firstRobot, secondRobot) {
  it.report("Win!")
}
```

In Kotlin, if a lambda is the *last* parameter of a function, it can be placed outside of the parentheses of a higher-order function.

In this example, the lambda being passed to `beginBattle()` is:

```
{ it.report("Win!") }
```

How do lambdas work?

When you defined the `onBattleEnded` lambda, it was compiled to the equivalent of the following Java code:

```
final class MainKt$main$onBattleEnded$1 extends Lambda
        implements Function1 {

  public static final MainKt$main$onBattleEnded$1 INSTANCE =
      new MainKt$main$onBattleEnded$1;

  public bridge invoke(Object arg0) {
    MainKt$main$onBattleEnded$1.invoke((Robot)arg0);
  }

  public final invoke(Robot robot) {
    robot.report("Win!");
  }
}
```

For every lambda, the Kotlin compiler generates a separate class, which extends an abstract class `Lambda` and implements an interface like `Function1`. The `Function1` interface is replaced by its alternatives (`Function0`, `Function2`, etc.) depending on the number of parameters of your lambda.

Take a look at the Kotlin source code of `Function1`:

```
/** A function that takes 1 argument. */
public interface Function1<in P1, out R> : Function<R> {
  /** Invokes the function with the specified argument. */
  public operator fun invoke(p1: P1): R
}
```

It's an interface with the single function `invoke()`, which receives a parameter of type `P1` and return type of `R`.

Let's find out what happens when you invoke your lambda in the equivalent Java code:

```
Function1 onBattleEnded =
    (Function1)MainKt$main$onBattleEnded$1.INSTANCE;
onBattleEnded.invoke(winner);
```

The lambda is converted to an instance of the generated `Lambda` sublass with the `Function1` type and its `invoke()` function is called and passed the arguments that were passed into the lambda.

Closures

Lambas (as well as local functions) act as **closures**, which means that they can access and modify variables defined outside of their own scope. Unlike Java, variables declared in the outer scope can be modified within the closure.

Take a look at the following example:

```
var result = 0

val sum = { a: Int, b: Int ->
  result = a + b
}

sum(5, 18)
```

The `result` value changes inside the `sum` lambda. Here's what happens under the hood in the equivalent Java code:

```
final IntRef result = new IntRef();
result.element = 0;
Function2 sum = (Function2)(new Function2() {
  public Object invoke(Object var1, Object var2) {
    this.invoke(((Number)var1).intValue(),
        ((Number)var2).intValue());
    return Unit.INSTANCE;
  }

  public final void invoke(int a, int b) {
    result.element = a + b;
  }
});
sum.invoke(Integer.valueOf(5), Integer.valueOf(18));
```

`IntRef` is a wrapper around the `result` variable, allowing you to access it inside the lambda:

```
public static final class IntRef implements Serializable {
  public int element;

  @Override
  public String toString() {
    return String.valueOf(element);
  }
}
```

There are wrappers available for all primitives and the `Object` base class.

You are already familiar with what is happening with `sum(5, 18)`. The Kotlin compiler generates an instance of `Function2` (as `sum` is a lambda with two parameters) and calls its `invoke()` function.

Extension functions

You learned about extension methods on classes in Chapter 14. Let's look at them again from the perspective of functional programming.

Sometimes you need to extend the functionality of a specific class. And, quite often, direct inheritance is not an option — your class could already extend another class, for example, or the required class isn't open for inheritance.

Take a look at the following example:

```
fun String.print() = System.out.println(this)
```

`String` is now a **receiver** type for the extension function.

You can use the function like this:

```
val string = "Hello world"
string.print()
```

The `String` class is final in Java, so you can't extend it. But, now you can call the new `print()` function on `String` instances. See what's generated from the above function:

```
public static final void print(@NotNull String $receiver) {
  System.out.println($receiver);
}
```

So, it's an ordinary function but, as a first argument, it implicitly *receives* an instance of the extended class on which this function was called. You can access it without any qualifiers or using `this` keyword.

It's time to further develop your battle robots. Create the following extension functions for `Random` in the `extensions.kt` file:

```
fun Random.randomStrength(): Int {
  return nextInt(100) + 10
}

fun Random.randomDamage(strength: Int): Int {
  return (strength * 0.1 + nextInt(10)).toInt()
}

fun Random.randomBlock(): Boolean {
  return nextBoolean()
}
```

You'll use these functions to calculate the strength of a robot, calculate the the damage it can do, and determine whether it can defend itself.

Update the `Robot` class to use the newly created functions:

```
private var random: Random = Random()

init {
  strength = random.randomStrength()
  report("Created (strength $strength)")
}

fun damage(damage: Int) {
  val blocked = random.randomBlock()

  if (blocked) {
    report("Blocked attack")
    return
  }

  health -= damage
  report("Damage -$damage, health $health")

  if (health <= 0) {
    isAlive = false
  }
}

fun attack(robot: Robot) {
  val damage = random.randomDamage(strength)
  robot.damage(damage)
}
```

You've added new functionality to the `Random` class without inheritance and used the new functionality within the `Robot` class.

Lambdas with receivers

Just as you can specify a receiver for an extension function, you can do so for a lambda as well.

Look at the `onBattleEnded` lambda parameter you created earlier and change its type from `(Robot) -> Unit` to `Robot.() -> Unit`. Note the subtle difference: all you added was `.()`. Here's what `beginBattle()` looks like now:

```
fun beginBattle(
  firstRobot: Robot,
  secondRobot: Robot,
  onBattleEnded: Robot.() -> Unit
) {
  var winner: Robot? = null
```

```
    battle(firstRobot, secondRobot)
    winner = if (firstRobot.isAlive) firstRobot else secondRobot
    winner.onBattleEnded()
  }
```

Now the type of the `onBattleEnded` lambda is `Robot.() -> Unit`. You invoke the lambda on the receiver `winner` using `winner.onBattleEnded()`.

Recall that an extension function implicitly receives an instance of the extended class. This means that you can still use this lambda in the following way:

```
onBattleEnded(winner)
```

However, using the lambda with receiver syntax `winner.onBattleEnded()` gives a clearer indication of which robot instance is handling the code within the `onBattleEnded` lambda.

Anonymous functions

Anonymous functions are more or less the same as ordinary ones, but they don't have a name. To invoke them, you need to assign them to a variable or pass them as an argument to another function. Consider the following snippet of code:

```
fun(robot: Robot) {
  robot.report("Win!")
}
```

This function can be used in the same way that you use regular functions — to invoke, pass as an argument, assign to a variable, etc.

Therefore, you can pass this function to the `beginBattle()` function instead of the lambda expression you used before:

```
val reportOnWin = fun(robot: Robot) { robot.report("Win!") }
Battlefield.beginBattle(firstRobot, secondRobot, reportOnWin)
```

You must reset your `beginBattle()` function signature to be the earlier version:

```
fun beginBattle(
  firstRobot: Robot,
  secondRobot: Robot,
  onBattleEnded: (Robot) -> Unit
)
```

You can also use a more concise form when passing in the anonymous function by omitting the type of a parameter if it can be inferred from the context:

```
Battlefield.beginBattle(firstRobot, secondRobot, fun(robot) {
  robot.report("Win!")
})
```

Returning from lambdas

If you use a regular `return` expression inside a lambda, you'll return to the call site of the outer function. That is, the `return` in the lambda also returns from the outer function.

Consider the code snippet below, where a lambda is passed to `forEach()`:

```
fun calculateEven() {
  var result = 0

  (0..20).forEach {
    if (it % 3 == 0) return

    if (it % 2 == 0) result += it
  }

  println(result)
}
```

You'll never get `result` printed as the `return` statement in the lambda stops the execution of `calculateEven()`. But if you only need to return from the lambda expression, you can use a **qualified return**:

```
fun calculateEven() {
  var result = 0

  (0..20).forEach {
    if (it % 3 == 0) return@forEach

    if (it % 2 == 0) result += it
  }

  println(result)
}
```

This way, as soon as an element is a multiple of three, the current iteration of the loop will be interrupted, and the next one will start. This behavior is similar to the use of a `continue` statement.

So the `result` variable will be equal to the sum of all *even* elements from 0 to 20, except for multiples of three.

The code above could also be rewritten in the following way:

```
fun calculateEven() {
  var result = 0

  (0..20).forEach loop@{
    if (it % 3 == 0) return@loop

    if (it % 2 == 0) result += it
  }

  println(result)
}
```

`loop` is just an explicit label, and you can use any of them to return to the place you need.

If you replace the lambda above with an anonymous function, you could use a regular `return` to return from the function and get the same result:

```
fun calculateEven() {
  var result = 0

  (0..20).forEach(fun(value) {
    if (value % 3 == 0) return

    if (value % 2 == 0) result += value
  })

  println(result)
}
```

Inline functions

Remember that, for each lambda that you create to pass to another function, the Kotlin compiler generates an appropriate class extending `FunctionN`. In some cases, that might not be a good solution, especially when you do it multiple times, as this will increase memory usage and have a performance impact on your application.

To avoid such behavior, you can mark your function with the `inline` keyword, which replaces the function call at the call site with the body of the function. Using `inline`, no additional classes are generated, and invocations of this function and received lambdas are replaced by their body.

Let's see how inlining works. Make the `beginBattle()` function `inline`:

```
inline fun beginBattle(
  firstRobot: Robot,
  secondRobot: Robot,
  onBattleEnded: Robot.() -> Unit
)
```

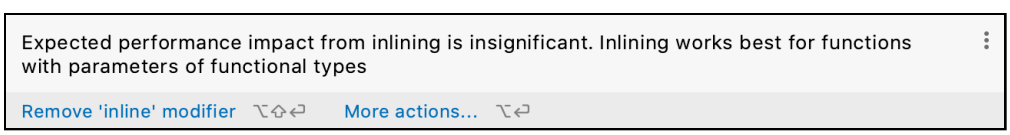
Now, the `main()` function body will be generated into the following equivalent Java code:

```
public static final void main(@NotNull String[] args) {
  Robot firstRobot =
    new Robot("Experimental Space Navigation Droid");
  Robot secondRobot =
    new Robot("Extra-Terrestrial Air Safety Droid");
  Battlefield this_$iv = Battlefield.INSTANCE;
  Robot winner$iv = (Robot)null;
  this_$iv.battle(firstRobot, secondRobot);
  winner$iv = firstRobot.isAlive() ? firstRobot : secondRobot;
  winner$iv.report("Win!");
}
```

As you can see, there are no invocations of the `beginBattle()` function; it's replaced by its body. The `onBattleEnded` lambda invocation also disappeared; now you can only see its body.

That may seem like a nice workaround for the overhead of Kotlin lambdas, and so it is. But this solution causes growth in the size of your generated code. You'll need to decide whether to inline your function or not based on a code size versus performance tradeoff.

If you try to inline a function which doesn't receive any lambdas as parameters, then inlining has a high probability of being useless; no extra classes get generated and there's no need to inline the function. In this case, you'll see the following warning in the IDE:



Expected performance impact from inlining is insignificant. Inlining works best for functions with parameters of functional types.

But if you're sure that inlining is necessary, use the `@Suppress("NOTHING_TO_INLINE")` annotation to hide the warning from the compiler:

```
@Suppress("NOTHING_TO_INLINE")
inline fun someFunction() {

}
```

Also, it's not a good idea to inline large functions, as it'll cause your generated code to grow significantly. Try to split the large function into several smaller functions, and inline only what's needed.

noinline

If you don't want some of the lambda parameters to be inlined along with the higher-order function, you can mark the lambda as `noinline`. A `FunctionN` instance will still be generated for `noinline` lambda:

```
inline fun someFunction(
  inlinedLambda: () -> Unit,
  noinline nonInlinedLambda: () -> Unit
)
```

If all lambda parameters of your function are marked with the `noninline` keyword, then inlining is probably pointless because of the reasons mentioned in the previous paragraph — the Kotlin compiler doesn't generate any extra classes, so it's not necessary to inline the function.

crossinline

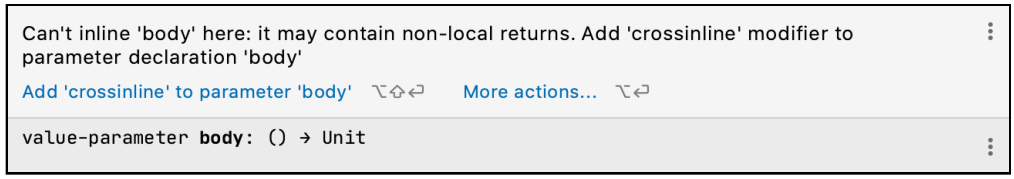
The `crossinline` keyword is used to mark a lambda parameter which shouldn't allow a non-local return (i.e., `return` without a label). This is useful when a function, which receives a lambda, will call it inside another lambda. In this case, it's not allowed to return from such a lambda. Take a look at the example below:

```
inline fun someFunction(body: () -> Unit) {
  yetAnotherFunction {
    body()
  }
}

fun yetAnotherFunction(body: () -> Unit) {

}
```

If you insert this snippet, you'll get the following compiler error:



Can't inline 'body' here: it may contain non-local returns. Add 'crossinline' modifier to parameter declaration 'body'

To avoid usage of a non-local return in the function parameter, and to make your project compile, you can use the `crossinline` keyword:

```
inline fun someFunction(crossinline body: () -> Unit) {
  yetAnotherFunction {
    body()
  }
}
```

After that, the compiler will issue a warning if you use a non-local return inside the `body` parameter:

```
fun oneMoreFunction() {
  someFunction {
    return
  }
}
```

'return' is not allowed here

Tail recursive functions

The last expression in a function is called the **tail call**. If, in some cases, the function gets called again in the tail call expression, this function is called **tail-recursive**. In Kotlin, you can mark such functions as `tailrec`, and the Kotlin complier will replace the recursion by an appropriate loop for the sake of performance optimization. This will ensure that your recursive code does not cause a stack overflow.

Add the `tailrec` keyword to the `battle()` function declaration:

```
tailrec fun battle(firstRobot: Robot, secondRobot: Robot)
```

To check how this works, take a look at the equiavlent Java code:

```
public final void battle(@NotNull Robot firstRobot, @NotNull
Robot secondRobot) {
  do {
    firstRobot.attack(secondRobot);
    if (!secondRobot.isAlive()) {
      return;
    }

    secondRobot.attack(firstRobot);
  } while (firstRobot.isAlive());
}
```

Using `tailrec`, the function is now based on the loop, and not on a recursive call.

Collections standard library

The use of standard library functions on collections that you saw in Chapter 10 are further examples of functional programming. The Kotlin standard library offers you a huge amount of useful functions for collection processing.

For example, define a list of robots which are expected to take part in the robot battle:

```
val participants = arrayListOf<Robot>(
  Robot("Extra-Terrestrial Neutralization Bot"),
  Robot("Generic Evasion Droid"),
  Robot("Self-Reliant War Management Device"),
  Robot("Advanced Nullification Android"),
  Robot("Rational Network Defense Droid"),
  Robot("Motorized Shepherd Cyborg"),
  Robot("Reactive Algorithm Entity"),
  Robot("Ultimate Safety Guard Golem"),
  Robot("Nuclear Processor Machine"),
  Robot("Preliminary Space Navigation Machine")
)
```

It's important to separate them into several categories by strength to avoid unfair fights. The first fight will be conducted for the top category of robots, so you need to find the strongest among them.

With Kotlin, you can do that by applying the following code:

```
val topCategory = participants.filter { it.strength > 80 }
```

That's much more concise than applying a loop. The `filter()` function is an example of a higher-order function, taking a function as its parameter. Of course, under the hood, the `filter()` function uses a loop to find all the appropriate elements in the list.

But that's not even the best thing about collection handling with functional programming. Quite often, it's necessary to apply several transformations at the same time:

```
val topCategory = participants
  // 1
  .filter { it.strength > 80 }
  // 2
  .take(3)
  // 3
  .sortedBy { it.name }
```

Here, you do the following:

1. Filter robots by their strength to find the strongest ones.
2. Take only first the three robots.
3. Sort the remaining robots by their names alphabetically.

Recall that functional programming is about functions without side effects. The code above fits this criterion. All of the functions you applied (`filter()`, `take()`, etc.) don't modify the original list in any way; they return a new list each time. Therefore, you can process the initial list as much as you need.

Infix notation

If a function is a member function or an extension function and receives only one argument, you can mark it with the `infix` keyword. That way you can invoke it without a dot and parentheses.

Mark the `attack()` function in the `Robot` class with the `infix` keyword:

```
infix fun attack(robot: Robot) {
  val damage = random.randomDamage(strength)
  robot.damage(damage)
}
```

Now, you can invoke it in the following way:

```
firstRobot attack secondRobot
```

Using the infix notation makes the code a bit more readable in certain cases.

Sequences

In Kotlin, you can use **Sequence** to create a lazily evaluated collection so that you can operate on collections of unknown size, which can be potentially infinite.

Here's an example of creating a sequence using `generateSequence()` from the standard library:

```
val random = Random()
// 1
val sequence = generateSequence {
  // 2
  random.nextInt(100)
}

sequence
  // 3
  .take(15)
  // 4
  .sorted()
  // 5
  .forEach { println(it) }
```

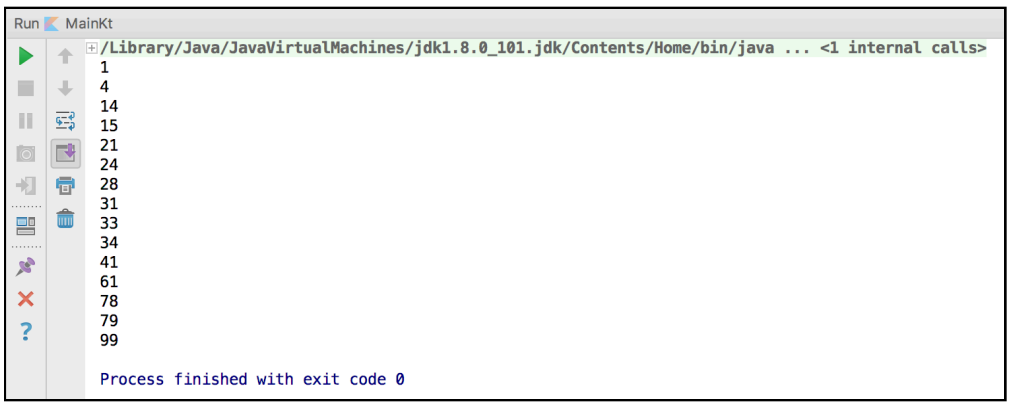
Here's what's going on in the code above:

1. You a create a sequence using `generateSequence()`, which receives a lambda of type `() -> T?` as an argument.
2. You return a random number from 0 to 100 from the lambda.
3. You take only the first 15 elements of the sequence.

4. You sort the elements.
5. You print each of them.

The lambda you passed to `generateSequence()` will be executed 15 times to evaluate the first 15 elements of the sequence.

If you run the app, you'll get a similar result:



Sequences can also be used to solve different kinds of mathematical tasks using functional programming. For example, you can find the factorial of 10 as follows:

```
val factorial = generateSequence(1 to 1) {
  it.first + 1 to it.second * (it.first + 1)
}

println(factorial.take(10).map { it.second }.last())
```

As the value of the factorial of N cannot be evaluated as a one-time operation, and you need to perform N - 1 multiplications, it's convenient to store the previous result to evaluate the next one.

In this case, you can use a `Pair` class, and you can use its `first` field to store the index, and the `second` to store the factorial for the current index. That way, when you calculate the factorial for the next index, you can access the value of the factorial for the previous one and multiply it by the incremented index. The above is an example of the technique known as **memoization**.

Challenges

1. Using the list of robot `participants` in the "Collections standard library" section, arrange a series of fights for the intermediate category of robots (i.e., their strength is around 40-80 points) between four participants. For example, you have the following list of robots: A, B, C, D. Therefore, you start from two battles A - B and C - D. The last fight will be conducted between the winners of those first two fights. **Note:** if you use random initial strengths, you may need to run the battle a few times to make sure you have enough intermediate participants.
2. Write a function to evaluate the first N elements of the **Fibonacci sequence** using memoization. Each of the elements of the Fibonacci is equal to the sum of the two previous ones. Start from 1, 1, 2, 3...

Key points

- Functional programming uses **first-class** functions, which can be passed as arguments, returned or assigned to variables.
- A **higher-order function** is a function that receives another function as a parameter and/or returns one.
- A **lambda** is a function literal defined in curly brackets, and can be invoked, passed to a function, returned or assigned to a variable.
- When you create a lambda, an implicit class is created that implements a `FunctionN` interface, where `N` is number of parameters that the lambda receives.
- Kotlin lambdas act as **closures**, with access variables defined in the outer scope of the lambda.
- Extension functions implicitly receive an instance of the extended class as the first parameter.
- **Lambdas with receivers** are similar to extension functions.
- Mark a lambda that shouldn't support a non-local return with the `crossinline` keyword.

- Use the `tailrec` keyword to optimize tail-recursive functions.
- Use the `inline` keyword to replace a function invocation with its body.
- If a function is a member function or extension function, and it receives only one argument, you can mark it with an `infix` keyword and call it without the dot operator or parentheses.
- Use **sequences** to create lazily evaluted collections.

Where to go from here?

Functional programming opens a world of possibilities, which are difficult to cover entirely in a single chapter. But to deepen your knowledge after understanding the basics, you can move on to more advanced concepts, such as function **composition**, **Either**, **Option**, **Try**, and more.

You can start by investigating the great library funKTionale (https://github.com/MarioAriasC/funKTionale), where you can find implementations of the concepts mentioned above, as they're not a part of the Kotlin standard library.

In the next chapter, you'll learn about the concept of **conventions** in Kotlin and see how they're used to allow **operator overloading**.